

Project Documentation: Live Weather Station

Ghile Patricia-Maria

January 19, 2026

Abstract

This project implements a standalone IoT Weather Station using the ESP32 microcontroller and the BME280 environmental sensor. The system functions as a modular web server capable of monitoring temperature, humidity, and atmospheric pressure in real-time. It features a responsive web interface, a weather prediction algorithm, and a circular buffer data logging system, all operating independently without the need for an external internet connection.

1 Introduction

The "Live Weather Station" is designed to provide accurate environmental data in a portable format. By leveraging the ESP32's dual-core processing power and built-in Wi-Fi capabilities, the device creates its own Access Point (AP). Users can connect to this network to view live data and historical trends directly on their smartphones or laptops via a generated web page.

2 System Architecture

The software is designed using a **modular architecture** to ensure code readability and ease of maintenance. The codebase is split into three distinct files:

- **main.ino**: The controller. Handles Wi-Fi, the Web Server, and data storage.
- **sensor.h**: The hardware abstraction layer. Handles I2C communication with the BME280.
- **design.h**: The frontend. Contains the HTML/CSS code for the user interface.

The overall data flow of the system is illustrated in Figure 1. The BME280 sensor acquires raw environmental data and transmits it to the ESP32 controller via the I2C bus. The ESP32 processes this information and hosts a web server. When a user device (Smartphone/Laptop) connects to the station's Wi-Fi network, the server responds to HTTP requests by delivering the real-time interface.

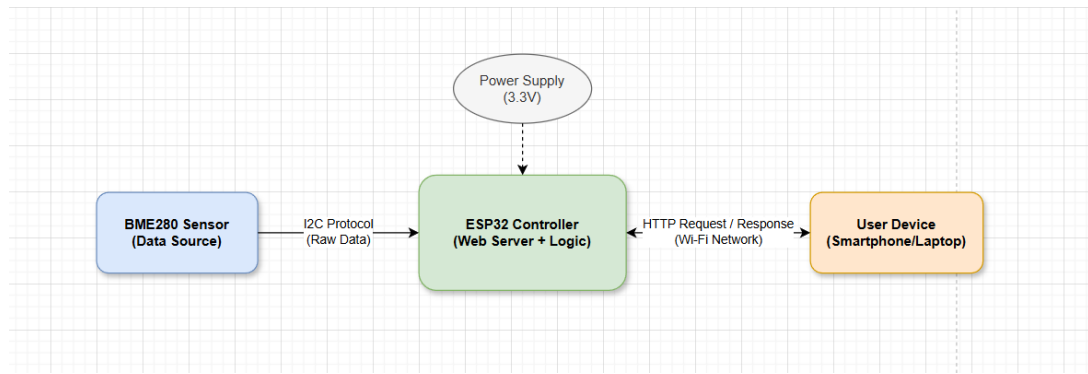


Figure 1: System Block Diagram: Data flow from Sensor to Web Interface

3 Hardware Configuration & Logic (sensor.h)

The ‘sensor.h’ module acts as a Hardware Abstraction Layer (HAL). Its primary role is to initialize the BME280 sensor via the I2C protocol and provide clean data to the main application.

3.1 Hardware Connections

To ensure proper communication, the BME280 sensor is wired to the ESP32 using the standard I2C pins. The specific wiring configuration is detailed in Table 1.

BME280 Pin	ESP32 Pin	Function
VCC	3V3	Power Supply (3.3V)
GND	GND	Ground connection
SCL (SCK)	GPIO 22	Serial Clock Line
SDA (SDI)	GPIO 21	Serial Data Line

Table 1: Wiring Diagram: BME280 to ESP32

The physical realization of the circuit is depicted in Figure 2. The image demonstrates the connections between the microcontroller and the sensor module, corresponding to the pinout configuration described above.

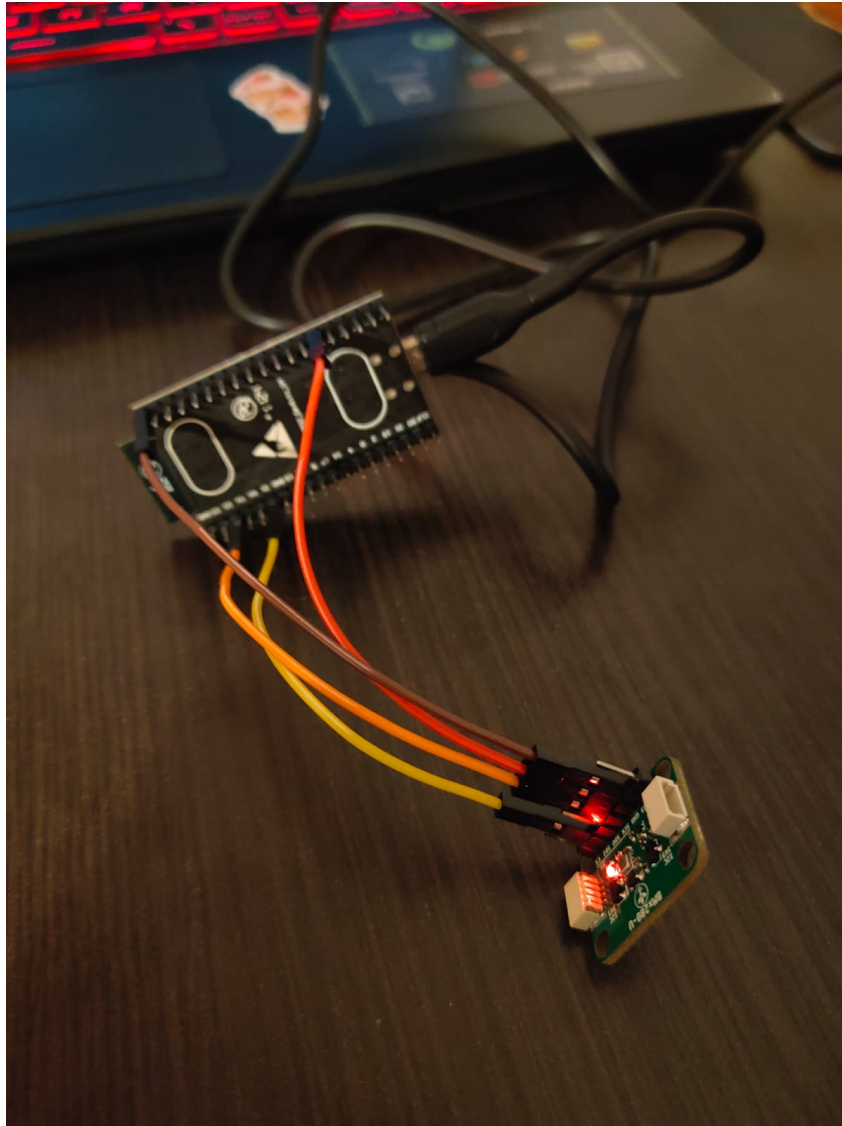


Figure 2: Physical Hardware Assembly of the Weather Station

3.2 Initialization Mechanism (Fail-Safe)

BME280 sensors can have one of two I2C addresses: ‘0x77’ (default for Adafruit) or ‘0x76’ (common for generic modules). To make the code robust, the ‘initSensor()’ function implements a fail-safe mechanism. It first attempts to connect to ‘0x77’. If this fails, it automatically retries connection at ‘0x76’, preventing runtime errors due to hardware variations.

```
1 void initSensor() {  
2   Serial.println("Initializing Hardware...");  
3   Wire.begin(21, 22);  
4  
5   if (!bme.begin(0x77)) {  
6     Serial.println("Not found at 0x77, trying 0x76...");  
7     if (!bme.begin(0x76)) {  
8       Serial.println("FATAL ERROR: Sensor not responding!");  
9     } else {  
10      Serial.println("Sensor connected at 0x76.");  
11    }  
12  }
```

```

12 } else {
13     Serial.println("Sensor connected at 0x77.");
14 }
15 }

```

Listing 1: Sensor Initialization Logic

3.3 Data Acquisition & Prediction

The module reads raw data and converts it into standard units. Specifically, pressure is read in Pascals (Pa) and divided by 100.0F to convert it to Hectopascals (hPa), which is the standard meteorological unit.

Additionally, a threshold-based algorithm predicts weather conditions:

```

1 String predictWeather(float u, float p) {
2     if (u > 80 && p < 1010) return "WARNING: Rain expected!";
3     if (u > 60) return "Cloudy.";
4     if (u < 40) return "Clear sky.";
5     return "Variable weather.";
6 }

```

Listing 2: Prediction Algorithm

4 Server & Data Logic (main.ino)

The ‘main.ino’ file acts as the central orchestrator. It sets up the Wi-Fi Access Point, manages the web server routes, and handles the temporary data storage algorithm.

4.1 Data Storage: Circular Buffer Algorithm

Since the ESP32 has limited RAM and no external database, I implemented a ****Shift Register (FIFO - First In, First Out)**** algorithm to store the history. When a new reading arrives:

1. The loop iterates backwards from the last element.
2. Each value is moved one position to the right (e.g., index 0 moves to index 1).
3. The newest reading is inserted at index 0.

This ensures that the array always contains the 10 most recent readings, discarding the oldest one automatically.

```

1 const int MAX_HISTORY = 10;
2 float tempHistory[MAX_HISTORY];
3
4 void saveToHistory(float t, float u, float p) {
5     for (int i = MAX_HISTORY - 1; i > 0; i--) {
6         tempHistory[i] = tempHistory[i-1];
7         humHistory[i] = humHistory[i-1];
8         presHistory[i] = presHistory[i-1];
9     }
10    tempHistory[0] = t;

```

```

11  humHistory[0] = u;
12  presHistory[0] = p;
13
14  if (historyCount < MAX_HISTORY) historyCount++;
15 }

```

Listing 3: Circular Buffer Logic

4.2 Web Server & Request Handling

The ESP32 is configured in **SoftAP Mode** (Software Access Point), meaning it broadcasts its own Wi-Fi network named "Weather Station" instead of connecting to a router.

When a client requests the root page (" / "), the `handleLive()` function is triggered. This function performs the following steps:

```

1 void handle_Live() {
2   float t = readTemperature();
3   float u = readHumidity();
4   float p = readPressure();
5
6   if (isnan(t)) {
7     server.send(200, "text/html", "<h1> Hardware Error!</h1>");
8     return;
9   }
10
11   saveToHistory(t, u, p);
12   String msg = predictWeather(u, p);
13
14   server.send(200, "text/html", getHTML(t, u, p, msg));
15 }

```

Listing 4: Request Handler

5 User Interface (design.h)

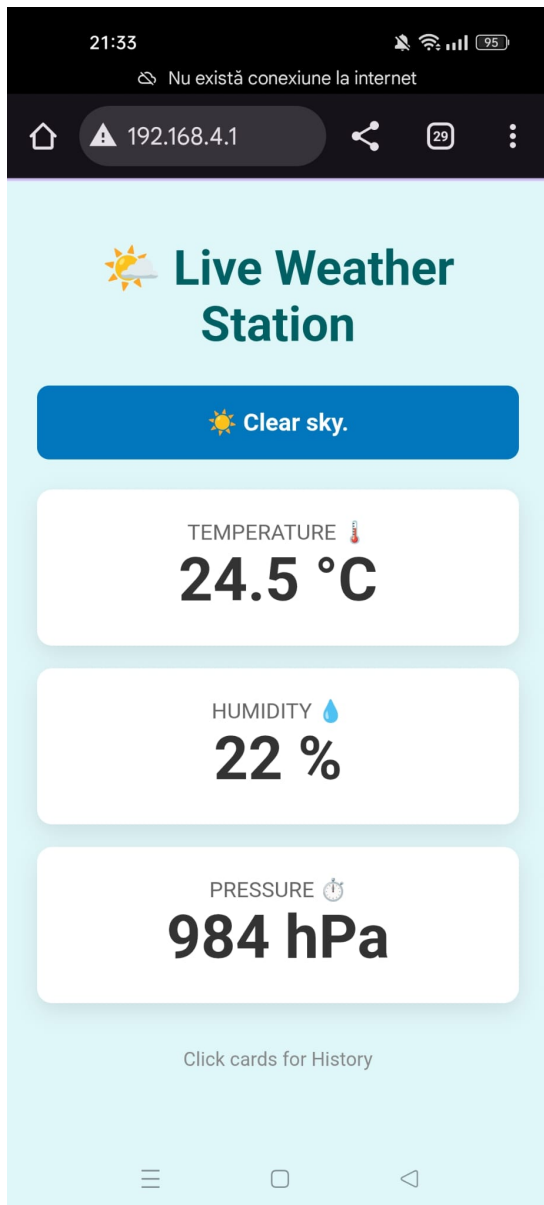
The `design.h` file acts as the Frontend. Instead of serving static files from an SD card, the HTML and CSS are stored as Strings within the program memory (Flash). This ensures the interface loads instantly.

Key Interface Features:

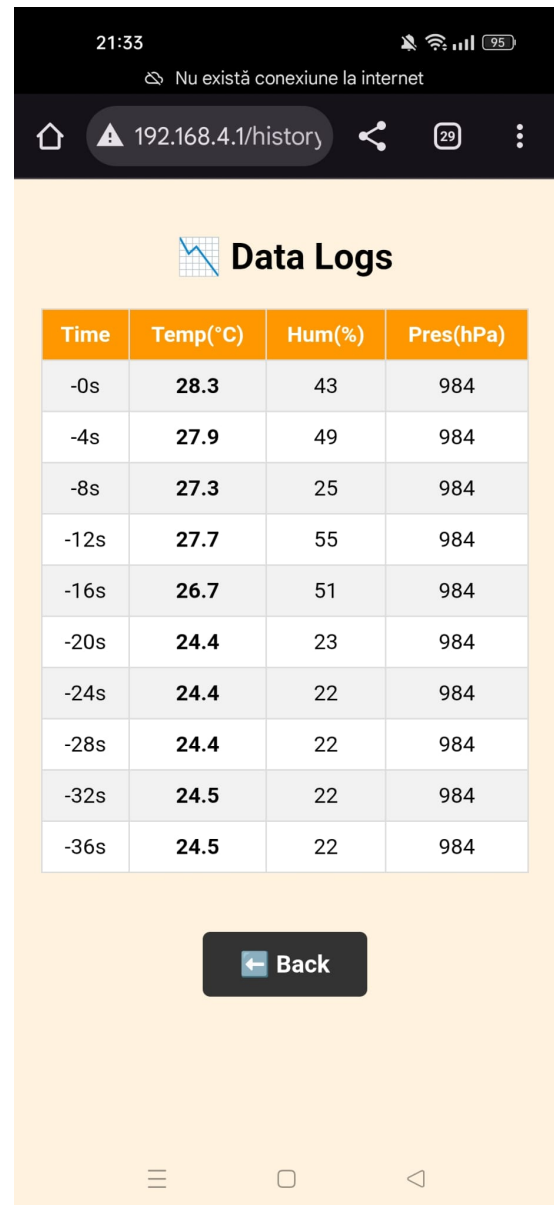
- **Responsive Design:** Uses CSS Grid/Flexbox to adapt to mobile screens.
- **Auto-Refresh:** A JavaScript script refreshes the data every 4 seconds via `location.reload()`.
- **Visual Feedback:** Color-coded cards for Temperature, Humidity, and Pressure.

5.1 Interface Screenshots

Below are the screenshots of the running application as seen on a mobile device connected to the ESP32's network.



(a) Live Dashboard



(b) Data History Log

Figure 3: Mobile Interface of the Live Weather Station

6 Conclusion

The Live Weather Station successfully integrates hardware sensors with software logic to create a functional IoT device. The modular code structure allows for easy expansion, while the self-hosted web server ensures functionality in remote areas without internet access.

References

- [1] Espressif Systems. (2024). *ESP32 Technical Reference Manual*.
- [2] Bosch Sensortec. (2023). *BME280: Combined humidity and pressure sensor data sheet*.
- [3] Arduino. (2024). *Arduino Language Reference*. Available online: <https://www.arduino.cc/reference/en/>
- [4] Adafruit Industries. (2024). *Adafruit BME280 Library*. GitHub Repository.
- [5] Danescu, R. *Proiectare cu Microprocesoare Course Page*. Technical University of Cluj-Napoca. Available online: https://users.utcluj.ro/~rdanescu/teaching_pmp.html
- [6] Espressif Systems. *ESP32 Wi-Fi & Bluetooth MCU*. Official Product Page. Available online: <https://www.espressif.com/en/products/socs/esp32>
- [7] ESPHome. *BME280 Temperature, Pressure and Humidity Sensor*. Documentation. Available online: <https://esphome.io/components/sensor/bme280/>